

Software Design Document (SDD)

Skin Cancer Detection App Using Machine Learning

Prepared by: Samuel Borges

Elif Dila Bayrak

Neva Ildem

Sophia Zubieta

Faculty Advisor & Client:

Dr. Zahra Nematzadeh

Department of Electrical Engineering and Computer Science

Florida Institute of Technology

Fall 2026

Table of Contents

1.0 Introduction	3
1.1 Purpose	3
1.2 Background and Objectives	3
1.3 Definitions and Acronyms	3
1.4 References	4
1.5 Document Overview	4
2.0 System Overview	5
2.1 User Account and Authentication	5
2.1.1 Authenticated User	5
2.1.2 Guest User	5
2.2 Image Capture and Upload	5
2.3 Image Analysis and ML Inference	6
2.4 Results Processing and Display	6
2.5 Analysis History and Data Management	6
3.0 System Architecture	6
3.1 Architectural Design	6
3.1.1 User Types Tier	7
3.1.2 User-Facing Machines Tier	7
3.1.3 Non-user-Facing Machines Tier	7
3.2 Decomposition Description	7
3.3 Design Rationale	9
4.0 Data Design	9
4.1 Data Description	9
4.2 Database Design	10
4.3 Data Dictionary	11
5.0 Component Design	12
5.1 Mobile Application Components	12
5.2 Server/API Components	13
5.3 Machine Learning Components	14
5.4 Data Management Components	15
6.0 Human Interface Design	17
6.1 Overview of User Interface	17
6.2 Screen Mockups	17
6.3 Screen Objects and Actions	18
7.0 Algorithm and Communication Design	18
7.1 Image Processing Pipeline	19
7.2 ML Classification Pipeline	19
7.3 Client-Server Communication	19
8.0 Security and Privacy Design	20
8.1 Authentication and Access Control	20
8.2 Secure Data Transmission	20
8.3 Data Storage and Protection	20
8.4 User Data Deletion	20
9.0 Performance and Optimization	21
9.1 Performance Targets	21
9.2 Optimization Strategies	21
10.0 Requirements Traceability Matrix	21
11.0 References	23
12.0 Appendices	23
12.1 Appendix A: System Glossary and Acronyms	23
12.2 Appendix B: System Error Messages	24
12.3 Appendix C: Screen Reference Index	24
12.4 Appendix D: Technology Stack	24

1.0 Introduction

1.1 Purpose

The architecture and system design of the Skin Cancer Detection App Using ML are described in this Software Design Document (SDD). This document outlines the application's structure and design to meet the functional and non-functional requirements defined in the Software Requirements Document (SRS).

The document outlines the main parts of the system and how the server, API, machine learning model, Android mobile application, and data storage components interact. The application's user interface design, data management, image processing, classification process, client-server communication, security and privacy measures, and other elements required for system implementation are also shown.

The purpose of this document is to provide a common understanding of the suggested system to the faculty advisor, the client, the project team, and other stakeholders. Additionally, this document will serve as a reference for the implementation and testing phases.

1.2 Background and Objectives

Early detection of potentially cancerous skin lesions can urge users to seek expert medical examination because skin cancer is a serious health concern. It is possible to develop applications that evaluate skin lesion photographs and provide users with preliminary information using ML and image classification. The Skin Cancer Detection App Using ML will serve as an assistive tool that lets users submit photos of skin lesions and receive an automated classification. The application is not meant to replace a certified healthcare professional's evaluation or offer a medical diagnosis. Instead, it is meant for informational and awareness purposes.

The project's primary goal is to create an Android mobile application that provides users with an easy way to submit images of skin lesions for ML analysis. The application will use a server/API architecture in which submitted images are analyzed by an ML model, and predictions on whether the lesion is benign or malignant are returned to the mobile application. Additionally, the system will provide user authentication and maintain a chronological record of images submitted and analyzed.

The design aims to provide a secure, organized, and easy-to-use system. The design will include mechanisms such as secure data transmission and storage, access control, and data deletion upon user request, as it contains sensitive user images and personal information. These objectives will shape the architectural, component, data, interface, communication, and security decisions throughout this document.

1.3 Definitions and Acronyms

- **AI:** Artificial intelligence. Computer systems designed to carry out tasks that usually require human intelligence.

- **API:** Application Programming Interface. A mechanism that allows a mobile application to communicate with a server for image analysis or other system functions.
- **CNN:** Convolutional Neural Network. A deep learning model commonly used for image classification.
- **GUI:** Graphical user interface. The visual interface where users interact with the application.
- **ML:** Machine learning. A branch of AI that allows computer systems to learn patterns from data and make predictions.
- **SDD:** Software Design Document. The document that explains the architectural components, interface, data, and other design aspects of a software system.
- **SRS:** Software Requirements Specification. A document that defines the functional and non-functional requirements of a software system.
- **UI:** User interface. The visual components that users interact with in the mobile application.
- **Authentication:** The process of verifying a user's identity before permitting access to their account and stored information.
- **Client:** The developed Android mobile application interacts with the server by sending requests and receiving responses.
- **Confidence Score:** A numerical value representing the confidence level of the ML model regarding the classification result.
- **Inference:** The process of using an ML model to generate a prediction for a new input image.
- **Preprocessing:** Operations such as resizing and normalizing that prepare the input image for an ML analysis.
- **Server:** A system that is not visible to the user, but receives requests from the mobile application and supports services such as data management and ML inference.

1.4 References

- IEEE Std 1016-2009, *IEEE Standard for Information Technology - Systems Design - Software Design Descriptions*, Institute of Electrical and Electronics Engineers (IEEE).
- Skin Cancer Detection App Using ML - Project Plan, Fall 2026.
- Skin Cancer Detection App Using ML - Software Requirements Specification (SRS), Fall 2026.
- Client/Faculty Advisor Requirements and Feedback, Dr. Zahra Nematzadeh, Department of Electrical Engineering and Computer Science, Florida Institute of Technology, Fall 2026.

1.5 Document Overview

The design choices of the Skin Cancer Detection App Using ML are defined in the remainder of this document. An outline of the system's main features is given in Section 2. The system

architecture and the relationships between its main parts are described in Section 3. The data design, which includes the database structure and data dictionary, is described in Section 4. The mobile application, server/API, machine learning, and data management components are covered in Section 5. The screen mockups and user interface design are shown in Section 6. The pipelines for image processing, machine learning classification, and client-server communication are covered in Section 7. Security and privacy design are covered in Section 8. System performance and optimization are covered in Section 9. The requirements traceability matrix is provided in Section 10, and the appendices and references are in Sections 11 and 12, respectively.

2.0 System Overview

The Skin Cancer Detection App is designed as a client-server mobile app system that is able to provide automated skin lesion detection and classification using machine learning. This system is designed to allow general users to capture or upload images of skin lesions, where it is then able to process these images through a normalized pipeline and submitted to a remote machine learning inference server. Using our trained Convolutional Neural Network/Transfer Learning Model, the system is able to evaluate the given image and return the classification result as Benign or Malignant, along with a prediction confidence percentage. An additional feature the application provides is the ability to historically track the data shown, along with safety disclaimers to ensure users are able to fully understand that the results provided are for awareness purposes only; they are not meant to be a diagnosis.

2.1 User Account and Authentication

Using the authentication module, user identity, account creation, and session persistence can be managed. This way, users have two operational pathways.

2.1.1 Authenticated User

An authenticated user is a user who has created an account with our system. This user first logs in or registers using their credentials. The authentication manager is then able to handle each session's tokens. It is also able to grant access to persistent cloud storage where past scans are logged with timestamps, and the images are kept.

2.1.2 Guest User

A guest user is able to bypass the registration process or the login process and is able to reach the core analysis features immediately. Guest sessions, however, operate without database storage; as a result, session data is discarded upon exiting the application.

2.2 Image Capture and Upload

The image capture and upload subsystem is the bridge between camera hardware and local device storage and the application's processing pipeline. The camera module interfaces with

Android's OS camera hardware. This is done to request the necessary operating system permission to capture high-resolution images of skin lesions in real time. The gallery importer is the interface that is compatible with Android's photo gallery and media library. This allows users to select existing images within the gallery to upload as the picture for the system to use. The image validation and pre-check module is the part that is able to analyze the raw input image to ensure it follows basic quality metrics before its upload. If the image fails, it prompts the user to rescan or reupload a different image.

2.3 Image Analysis and ML Inference

The image analysis pipeline handles image preprocessing and remote model execution. This module standardizes incoming images to match the desired format of the training dataset (ISIC Archive). Operations include cropping, resizing, and color normalization. The API Communication Gateway is then able to transmit the preprocessed image payloads securely over HTTPS to the remote machine learning server. The inference engine is the part that executes the trained CNN/Transfer Learning model on the non-user-facing server to evaluate features and to output the probability scores of Benign vs. Malignant.

2.4 Results Processing and Display

This subsystem formats raw output probabilities into very clear visual feedback for the users. The confidence visualizer transforms the raw model softmax outputs into user-friendly and easy-to-understand confidence percentages. There will also be a confidence presenter; this feature will display the primary prediction as binary or malignant. There will also be a safety and next steps logic; this feature will evaluate risk thresholds. This way, if there is a high-confidence malignancy or low-confidence benign output, the system displays targeted safety guidance and urges the user to seek a professional opinion.

2.5 Analysis History and Data Management

This feature is only for authenticated users who have an account with our system; this subsystem manages past image sessions using a data manager. This is able to package the analyzed image, provide a timestamp, and its classification, along with its confidence score, into a user record. This allows the user to refer back to the history. The cloud database interface coordinates these secure operations with HTTPS to the backend database; this allows users to view a chronological feed of past scans or manually delete stored entries.

3.0 System Architecture

3.1 Architectural Design

Our system will be made up of a three-tier client-server architecture; this will include three physical and operational tiers.

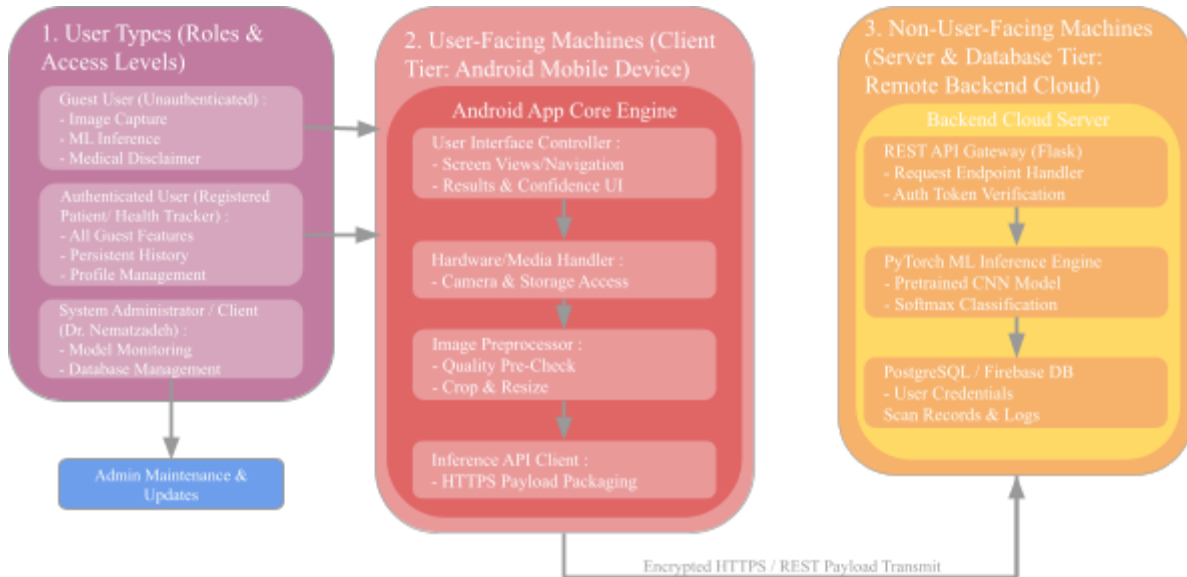


Figure 1. System Architectural Design

3.1.1 User Types Tier

There are three types of users: authenticated users, guest users, and administrators. Guest users have access to image upload, and the model preprocesses, analyzes, and provides safety disclaimers without saving sessions. Authenticated users have the same features as guest users, with the addition of cloud history, management of current and previous sessions, and record deletion. Administrators oversee the model backend updates and help with the integrity and health of the system.

3.1.2 User-Facing Machines Tier

This tier deals with the client's needs. This is the portion that will execute on the Android smartphone. In addition, this tier holds the UI for the application, along with the HTTPS REST network requests.

3.1.3 Non-user Facing Machines Tier

This tier is the one that the user does not interact with. This has the job of hosting the web framework and database and operates on a remote cloud server. Here is where the software modules handle the REST requests, and where they are able to execute the authentication process for logging in and maintaining the database.

3.2 Decomposition Description

The software system can be decomposed into six object-oriented modules and classes.

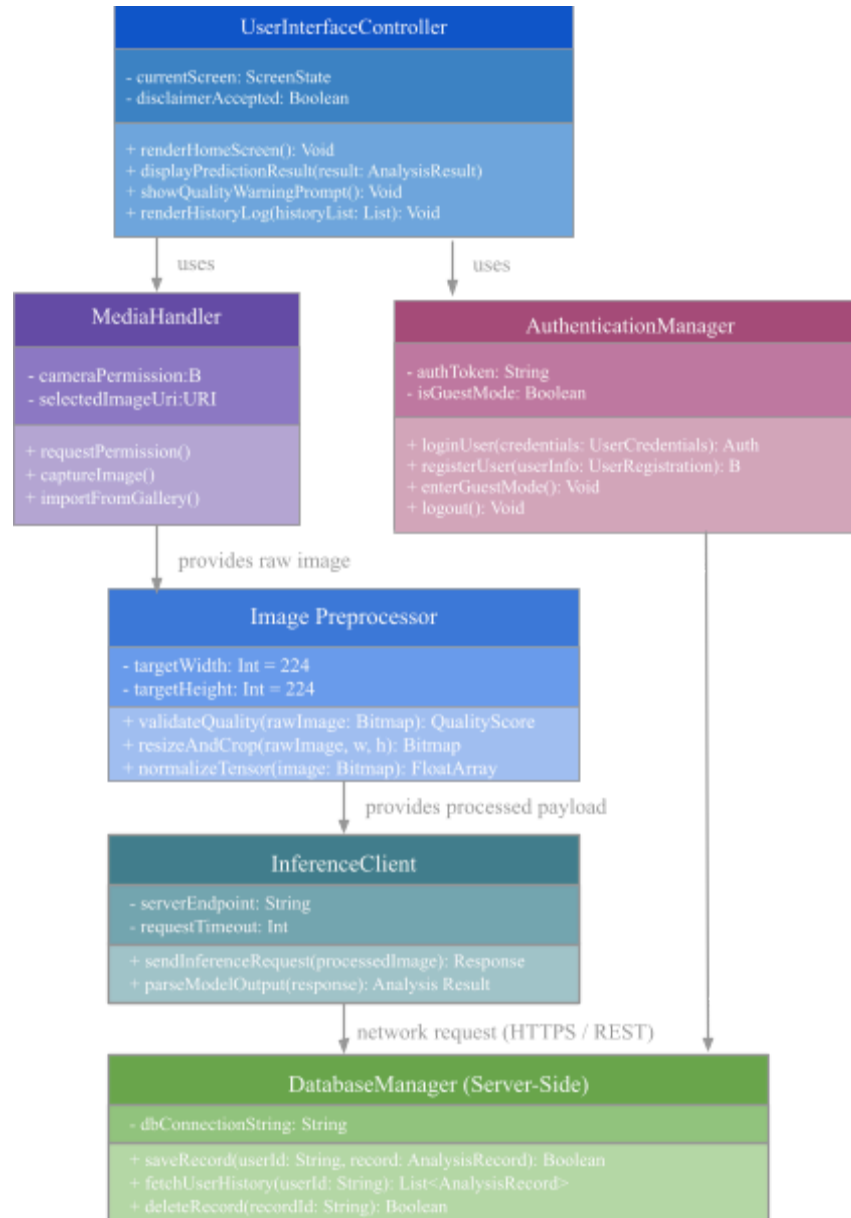


Figure 2. Decomposition Diagram

1. **UserInterfaceController module** - this is in charge of screen navigation, along with validation of input, result rendering, confidence scores, and display of medical disclaimers.
2. **AuthenticationManager module** - this module handles the encryption of the users' and administrators' credentials. Also, this module creates tokens, handles API requests dealing with login and registration, and flags guest sessions.
3. **MediaHandler module** - this module is able to interact with Android OS hardware to request permission to use the camera to be able to capture images and select images from the Android OS storage.

4. **ImagePreprocessor module** - this module normalizes images that are chosen or captured for the model system to be able to process. Normalizing methods include cropping transformations, RGB channel normalization, and image sharpness and blurriness metrics.
5. **InferenceClient module** - this module takes the normalized image data and packages it into JSON HTTP payloads; this establishes REST communication between the backend ML server and breaks down into prediction scores.
6. **DatabaseManager module** - this module is part of the user's side of the system. This allows for secure storage of private images and data, query retrieval, and the deletion of user accounts. In addition, it is able to control the deletion process of timestamps, confidence scores, and image metadata.

3.3 Design Rationale

1. **Client-Server Split for ML Inference vs Local Edge Execution** - Our system will have a client-server architecture. By using a non-server-facing remote server, our system will have a reduced mobile application size; this will also avoid significant battery drain on the user's cellular device and allow for future updates on the system to happen without needing the user to redownload the application.
2. **Client-Side Preprocessing Prior to API Transmissions** - Our system will resize and normalize images on the user's cellular device before sending them over the network. The system will run this way to reduce the image byte size that will need to be sent over the network; this minimizes the user's bandwidth consumption while ensuring fast response time even while on slow network connections
3. **Optional Guest Mode** - Our system will provide users with an option to use guest mode instead of having to log into their account or register to create an account. By allowing guest entry, it allows for quick skin checks while maintaining a difference between data storage and temporary session memory.
4. **HTTPS Encryption and Token-Based Authentication** - Our system will implement token-based authentication and will enforce encrypted network protocols, as users will be trusting our system with their sensitive data and images. Skin health images and historical scan records require data privacy, and by implementing these methods, our system protects API endpoints against unauthorized access.

4.0 Data Design

4.1 Data Description

To support user accounts, skin lesion analysis, and analysis history, the system will handle a variety of data formats. While data created during a guest session will be used only temporarily for analysis and will not be added to the user's analysis history, data associated with authenticated users will be permanently retained.

The system will keep track of the account information required for both authentication and identifying the user's stored records for those who have been authenticated. An analysis record with the provided image, the benign or malignant classification result, the ML model's confidence score, and a timestamp showing the analysis's execution will be generated for each completed skin lesion analysis. The authenticated user's account will be linked to these records, enabling the retrieval and chronological display of earlier analyses.

Images of skin lesions will be sent to the server for ML inference and preprocessing. Until the user decides to remove the recorded entry or related account data, the image and its related analysis data may be kept in the user's history for authenticated users. The image and analysis data will be handled only as temporary session data for guest users; they won't be able to retain their results permanently after the session ends.

4.2 Database Design

The database will store account information and analysis metadata for authenticated users. The actual images of skin lesions will be directly stored in the database. Instead, the images will be stored separately, and each analysis record in the database will reference the corresponding image. This distinction will reduce the amount of data stored directly in the database and help it manage information more effectively.

The database will consist of two main sections: *User* and *Analysis Record*. The User section will represent each user and include the information needed to identify and authenticate an account. Each user will be assigned a unique key called *user_id*.

The Analysis Record section will represent a completed analysis of a skin lesion. Each entry will be assigned a unique key, *analysis_id*, and a *user_id* that links the analyzed image to the corresponding user. Each entry will also include an *image_reference*, which is a reference to the image's location in the image storage, along with the classification result, confidence score, and the timestamp of the analyzed image.

User and Analysis Record sections will have a one-to-many relationship. An authorized user may have multiple analysis records, whereas each analysis corresponds to a single user. The guest users will not exist in the permanent database. Their images and analysis results are temporary and will be deleted after their session ends.

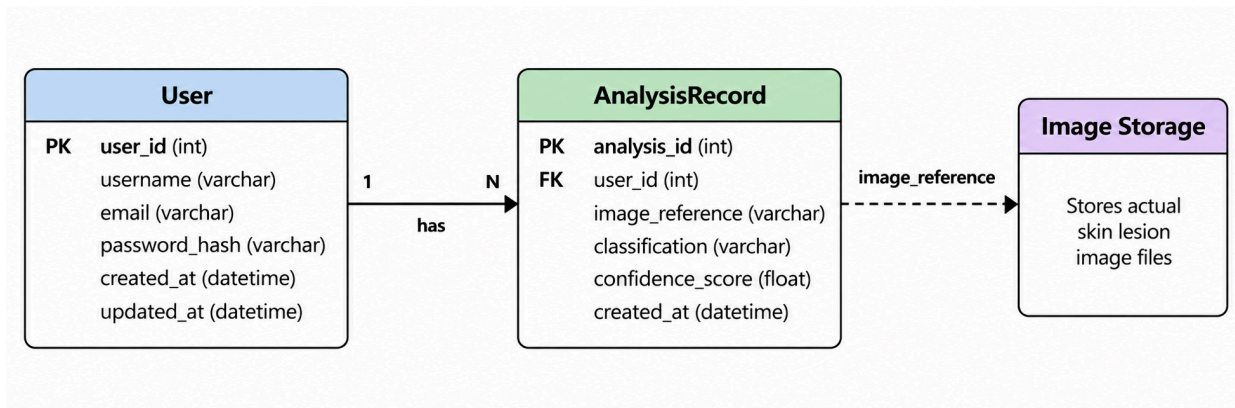


Figure 3. Database Design

4.3 Data Dictionary

The data dictionary defines the main data elements that will be used. Actual skin lesion images are stored separately and referenced by analysis records, while the database contains information on analysis entries for authorized users. Sessions for guest users are temporary and therefore are not included in the permanent database.

Table 4.1 - User Data

Data Element	Type	Key	Description
<i>user_id</i>	Integer	PK (Primary Key)	Unique key assigned to each authenticated user.
<i>username</i>	String	-	Username associated with a user's account.
<i>email</i>	String	-	Email associated with a user's account.
<i>password_has</i>	String	-	A secure representation of a password used for authentication.
<i>created_at</i>	Date & Time	-	Date and time the user account was created.
<i>updated_at</i>	Date & Time	-	Date and time of the last activation of a user account.

Table 4.2 - Analysis Records Data

Data Element	Type	Key	Description
<i>analysis_id</i>	Integer	PK (Primary Key)	Unique key associated with each completed analysis.
<i>user_id</i>	Integer	FK (Foreign Key)	Refers to the user account associated with the respective analysis.
<i>image_reference</i>	String	-	Reference to the respective image of an analysis that is stored separately.
<i>classification</i>	String	-	ML classification result: Benign or Malignant.
<i>confidence_score</i>	Float	-	Confidence value regarding the ML's

			prediction result.
<i>created_at</i>	Date & Time	-	Date & time of the performed skin lesion analysis.

The FK (Foreign Key) *user_id* in the Analysis Record links each stored analysis result to the corresponding authorized user. *image_reference* links all the analysis information to the actual skin lesion image stored in a separate storage. Guest users do not contribute to the permanent database, as their images and corresponding analysis results are deleted after their session ends.

5.0 Component Design

5.1 Mobile Application Components

The components of the mobile application will run on the user's Android device and will serve as a primary interface between the user and the remaining components of the system. These components will manage user interactions, authentication, image capture and upload, communication with the server/API, and the display of analysis results. The mobile application will support both authenticated and guest users, while features such as analysis history will be available only to authenticated users.

- **User Interface Controller:** Will manage the application's screen and navigation structure. Will respond to user actions and control the transitions between login, home page, image selection, analysis result, history, settings, and information screens. It will also provide the classification results, confidence scores, error messages, and the legal disclaimer.
- **Authentication Manager:** Will manage interactions regarding the account creation, login, logout, and user accounts. It will distinguish between authenticated and guest users and send authentication requests to the server/API. While authenticated users can access their saved analysis history, guest users can only access the image processing functionality and do not have access to their analysis history.
- **Media Handler:** Provides access to the Android device's camera and photo library. Will request the necessary device permissions and enable the user to capture a new skin lesion image or upload an image already on the device. The selected image will be processed and analyzed using the ML pipeline.
- **Inference Client:** Manages the communication between the mobile application and the server/API for the skin lesion analysis. Will send image data via a secure HTTPS request and receive the classification result and confidence score from the server. Also, it will detect communication errors and will provide an appropriate message when the server cannot be reached.

Table 5.1 - Proposed Mobile Application Component Interfaces

Component	Interface Method (Example)	Function
-----------	----------------------------	----------

User Interface Controller	<i>navigateToScreen()</i>	Navigates between application screens.
User Interface Controller	<i>displayResult()</i>	Displays classification and confidence score.
Authentication Manager	<i>login()</i>	Submits user credentials for authentication.
Authentication Manager	<i>logout()</i>	Ends an authenticated user session.
Authentication Manager	<i>continueAsGuest()</i>	Starts a guest session without permanent storage.
Media Handler	<i>captureImage()</i>	Opens the device's camera to capture an image.
Media Handler	<i>selectImage()</i>	Opens the device's photo library to select an image.
Inference Client	<i>submitImage()</i>	Sends image data to the server/API for analysis.
Inference Client	<i>receiveResult()</i>	Receives the classification response from the server.

5.2 Server/API Components

The server/API components will be in the non-user-facing part of the system and enable communication among the Android mobile application, the ML model, and the persistent data storage components. The server will process requests from the mobile application, handle authentication and analysis, communicate with the related back-end systems, and send responses to the mobile application. HTTPS will be used for the communication between the mobile application and the server/API.

- **API Request Handler:** Receives requests from the mobile application and sends them to the server-side component. Will process requests such as authentication, image analysis, analysis history, and deletion of saved entries.
- **Authentication Service:** Will process account creation and login requests from the application. Will authenticate user information and will manage authentication tokens to identify authenticated sessions. Provide guest users with the image analysis feature without creating permanent records.
- **Analysis Service:** Will receive sent images and will send them to the ML model for inference. After the ML model processes the image, it will provide a prediction of benign or malignant along with a confidence score, returning the result to the application.

- **Data Access Service:** Will manage interactions between the database and the separate image storage. Will support features such as storing analysis results and their related images for authenticated users, and deleting entries. The analysis results for guest users will not be stored permanently.

Table 5.2 - Proposed Server/API Component Interfaces

Component	Interface Method (Example)	Function
API Request Handler	<i>handleRequest()</i>	Receives and directs requests from the mobile application.
Authentication Service	<i>registerUser()</i>	Processes a new user account creation request.
Authentication Service	<i>authenticatedUser()</i>	Verifies the credentials of a user.
Analysis Service	<i>processAnalysis()</i>	Sends submitted image metadata to the ML model for analysis.
Analysis Service	<i>returnPrediction()</i>	Returns the classification result and the confidence score to the mobile application.
Data Access Service	<i>saveAnalysis()</i>	Stores the analysis results and image reference for authenticated users.
Data Access Service	<i>getAnalysisHistory()</i>	Retrieves the stored analysis results for authenticated users.
Data Access Service	<i>deleteAnalysis()</i>	Deletes the selected stored analysis entry and the corresponding image.

5.3 Machine Learning Components

The machine learning components will be responsible for preparing skin lesion images for the trained model's analysis, producing classification predictions, and forwarding the results and confidence scores to the server/API. The final ML architecture is still to be decided. Candidate models will be selected based on an analysis of recent studies on skin lesion classification, and the selected model will then be trained and evaluated on the skin lesion dataset. The final model will be selected by comparing candidate models using evaluation metrics and assessing their ability to generalize to unseen data.

- **Image Preprocessor:** Will preprocess skin lesion images in accordance with the machine learning model's input specifications. Resizing, normalization, and other picture preparation methods decided upon during model development are examples of preprocessing. Based on the results of the literature review, techniques for resolving class imbalance and data augmentation will also be examined through training.

- **Model Manager:** Will oversee both the final trained model utilized by the application and the candidate machine learning models used during development. The literature review and experimental assessment of the skin lesion dataset will be used to choose candidate architectures. Additionally, loading the trained model for inference will be supported by the component.
- **Inference Engine:** Will run the trained model to produce prediction probabilities after receiving a preprocessed image of a skin lesion. Instead of using the Android device directly, the inference process will be carried out on the distant server.
- **Prediction Processor:** Will deliver the benign or malignant categorization along with its confidence score after interpreting the inference engine's output. After that, the results will be sent back to the Analysis Service before being shown to the user.

Table 5.3 Proposed Machine Learning Component Interfaces

Component	Interface Method (Example)	Function
Image Preprocessor	<i>preprocessImage()</i>	Prepares an input image according to the specifications of the chosen model.
Image Preprocessor	<i>validateImage()</i>	Determines if the image is appropriate for machine learning analysis.
Model Manager	<i>loadModel()</i>	Loads the trained model of choice for inference.
Model Manager	<i>getModelInfo()</i>	Gives details about the inference model that is currently in use.
Inference Engine	<i>runInference()</i>	Uses a preprocessed image of a skin lesion to run the trained model.
Prediction Processor	<i>getClassification()</i>	Converts the model output into a benign or malignant classification.
Prediction Processor	<i>getConfidenceScore()</i>	Returns the confidence score associated with the prediction.

5.4 Data Management Components

The data management components will manage information about authenticated users, including account information, analysis records, and stored skin lesion images. While information about accounts and analyses is stored in the database, the actual image files will be stored in a separate location and referenced by their corresponding analysis records. The data generated for guest users will not be stored permanently.

- **Database Manager:** Will manage interactions in the database for authenticated users. It will support the creation, calling, receiving, and deletion of user and analysis records, and it will also link each user to their corresponding analysis history.
- **Image Storage Manager:** Will manage the storage and calling of the skin lesion image files. Each stored image will be referenced with *image_reference* in the analysis record before it is permanently saved in the database.
- **Analysis History Manager:** Will provide the analysis records of an authenticated user and will provide the required information for these entries to be viewed chronologically. Each entry record will include the corresponding image, classification result, confidence score, and the timestamp.
- **Data Deletion Manager:** Will manage the process of deleting user data. When an analysis record is deleted, the corresponding analysis information and the related image will be removed. Deleting an account also removes a user's saved account information, analysis records, and related images.

Table 5.4 - Proposed Data Management Component Interfaces

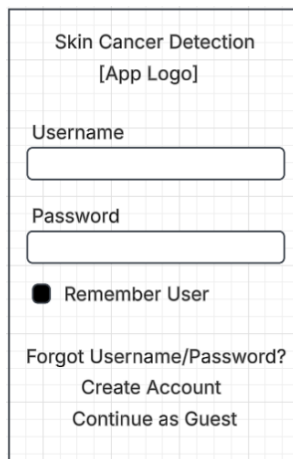
Component	Interface Method (Example)	Function
Database Manager	<i>createRecord()</i>	Creates a new database record.
Database Manager	<i>retrieveRecord()</i>	Retrieves stored data from the database.
Database Manager	<i>deleteRecord()</i>	Deletes a selected data entry.
Image Storage Manager	<i>storeImage()</i>	Saves a skin lesion image in a separate image storage.
Image Storage Manager	<i>retrieveImage()</i>	Retrieves an image using the saved reference.
Image Storage Manager	<i>deleteImage()</i>	Deletes an image from the image storage.
Analysis History Manager	<i>getHistory()</i>	Gets the saved analysis history of an authenticated user.
Data Deletion Manager	<i>deleteAnalysisData()</i>	Deletes an analysis record and its related image.
Data Deletion Manager	<i>deleteUserData()</i>	Deletes the saved account and all related data of an authenticated user.

6.0 Human Interface Design

6.1 Overview of User Interface

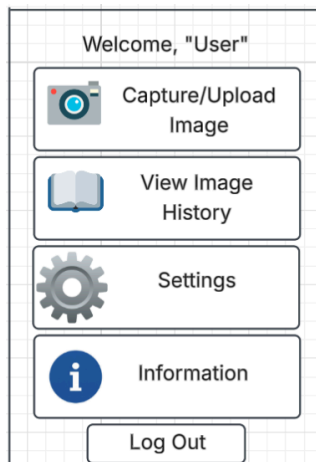
The overall goal is for the app to provide a simple, intuitive interface that allows users to easily navigate through the app. After the user downloads and opens the app, it will display a login page, giving the user the option to log in or sign up with an existing account (see figure in section 6.2). When the user logs in, the home screen will be displayed, offering the options to upload an image, view image history, and more. After the user uploads an image of the skin lesion, after the image is preprocessed and analysed, the app will display the prediction with a confidence level (see the figure in section 6.2).

6.2 Screen Mockups



A wireframe mockup of a user login screen. At the top, it says "Skin Cancer Detection" followed by "[App Logo]". Below this are two input fields: "Username" and "Password". Under the password field is a checkbox labeled "Remember User". At the bottom, there is a link "Forgot Username/Password?", a "Create Account" button, and a "Continue as Guest" button.

Figure 3. User Login Screen



A wireframe mockup of a home screen. At the top, it says "Welcome, 'User'". Below this are four main action buttons, each with an icon and text: "Capture/Upload Image" (camera icon), "View Image History" (book icon), "Settings" (gear icon), and "Information" (info icon). At the bottom is a "Log Out" button.

Figure 4. Home Screen

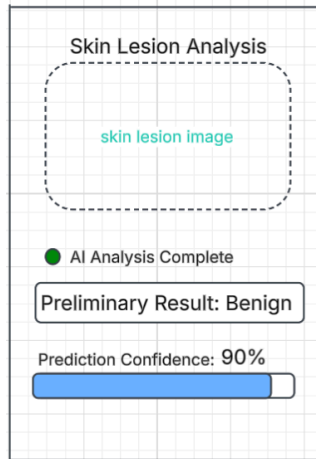


Figure 5. Skin Lesion Analysis Screen

6.3 Screen Objects and Actions

Screen object 1: Capture/Upload Image

Action: Opens the image selection screen, where the user can capture a new image or select an image from the device gallery.

Screen object 2: View Image History Action

Action: Opens the history screen, where users with an account can view previously saved images and analysis results.

Screen object 3: Settings

Action: Opens the Settings screen, where the user can modify available application and account settings.

Screen object 4: Information

Action: Opens the Information screen, which provides information about the application, image guidelines, safety, limitations, and medical resources.

7.0 Algorithm and Communication Design

In this section, we will describe how images will be processed and analyzed by the machine learning model and transferred between the mobile application and the server. The main process starts when the user uploads a picture or takes a picture of skin lesions through the app. The image is prepared through preprocessing, sent to the machine learning model for classification, and the prediction result is returned to the user.

7.1 Image Processing Pipeline

The order of the pipeline goes as follows: original image, resize, process into a model-ready image. Images that are shared by the app's users might have differing size marks, image resolution, lighting, or overall quality. Before we may analyze these images using our machine learning model, they will be sorted into a consistent format. Preprocessing will be used to classify the images.

Normalization will adjust pixel values of each image into the format best fit for the model. There will be an image check process to test if the image is blurry or unsuitable for app analysis. The user might be asked to submit another image. There will be a resizing component so that the image matches the dimensions that are required by the machine learning model. The app might also use noise reduction for the images that it deems necessary.

Python, OpenCV, and Keras are to be used for image preprocessing or augmentation. During the model development phase, techniques may be used to create more variety for the training data, such as flipping the image, rotation, brightness or contrast adjustments, and zooming.

7.2 ML Classification Pipeline

The order of the classification pipeline goes as follows: preprocessed image, ML model, benign/malignant prediction, confidence score, and display of results. The team will train and compare several available deep learning models specifically using skin lesion datasets: ISIC and HAM10000. Convolutional Neural Network (CNN) is being considered as an approach, as well as transfer learning models such as ResNet50, EfficientNetB0, and MobileNetV2.

Once the models have been completely trained, their performance may be measured by scoring accuracy, precision, recall/sensitivity, F1 score, and confusion matrices. This will allow the team to compare different models and better understand which one is appropriate for this type of application. Limiting the possibility of false negatives will be an essential consideration. Labeling a malignant lesion incorrectly and classifying it as benign would give the user misleading information.

After the most fitting model is chosen, we will pass a preprocessed image through the trained model to test the analysis. The model then classifies the skin lesion as benign or malignant and provides a confidence level for its prediction. The result of this confidence level will then be presented to the user.

7.3 Client-Server Communication

The order of the client-server communication flow goes as follows: mobile app, server/API, ML model, prediction and confidence, mobile app. The mobile application acts as the **client** and communicates with the server through the API. When the image is submitted into the app, it will

be sent through the server to be analyzed by the machine learning model. The prediction as well as the confidence score will be returned to the application to be displayed to the user.

If the server is unable to be reached, the application will display the error message “Server Unable to be Reached” rather than providing a prediction.

8.0 Security and Privacy Design

8.1 Authentication and Access Control

The application will provide authentication for users who choose to create an account. Authenticated users may only access the data associated with their account by entering the correct username and password.

Users who choose to navigate the application as a guest will not have their activity history saved after the session ends. User data will only be stored for users with a created account.

Users will be required to re-enter their account credentials each time the application is closed. If the “Remember Me” option is enabled, the user will remain signed in automatically.

8.2 Secure Data Transmission

Communication between the mobile application and the API will use HTTPS with TLS encryption to protect data while it is being transmitted. This will help prevent unauthorized parties from viewing account information, images, analysis results, or other user data during transmission.

8.3 Data Storage and Protection

The application will enforce a secure database backend that can store sensitive user metadata and image assets. All access points require explicit authentication, ensuring complete isolation of user data.

8.4 User Data Deletion

Because user data will be associated with a created account, deleting an account will result in stored data being fully and permanently deleted to protect privacy information by executing an automated, end-to-end hard delete workflow.

9.0 Performance and Optimization

9.1 Performance Targets

The performance targets aim to establish goals that will ensure that the app is reliable and performs its main operations.

- **Analysis Time:** The time of preprocessing and analysis should be less than 5 seconds before displaying the prediction.
- **Prediction Accuracy:** The confidence level should be precise, and the CNN should accurately predict the skin lesion type.
- **Navigation Response:** For each screen transition in the app, it should respond within 1 second under normal conditions.
- **Battery consumption:** The app should minimize battery consumption.
- **Analysis explanation:** The app should provide an explanation that is relevant to the image that was processed.

9.2 Optimization Strategies

When exploring optimization strategies, multiple strategies can be implemented, but the strategies below were selected as potential options due to easy implementation and efficiency:

- **Image Preprocessing Optimization:** Using preprocessing techniques like resizing and normalization to reduce computational workload.
- **CNN Optimization:** Using a lightweight CNN architecture or a mobile-optimized model such as MobileNetV2.
- **Efficient Data Management:** Store metadata separately from images to reduce database size.
- **Battery Optimization:** Performing operations like compressing images locally before transmission to reduce network usage.

10.0 Requirements Traceability Matrix

The Requirements Traceability Matrix connects the requirements defined in the Software Requirements Specification to the corresponding sections of the Design Document. This helps ensure that each major system requirement is addressed in the system design.

REQUIREMENT	REQUIREMENT DESCRIPTION	DESIGN SECTION
FR-1	Input of an Image	2.2 Image Capture and Upload, 5.1 Mobile Application Components, 6.3 Screen Objects and Actions

FR-2	Image Preprocessing	2.3 Image Analysis and ML Inference, 5.3 Machine Learning Components, 7.1 Image Processing Pipeline
FR-3	Machine Learning Model Training	5.3 Machine Learning Components, 7.2 ML Classification Pipeline
FR-4	Machine Learning Model Evaluation	5.3 Machine Learning Components, 7.2 ML Classification Pipeline, 9.1 Performance Targets
FR-5	Skin Lesion Prediction	2.3 Image Analysis and ML Inference, 5.3 Machine Learning Components, 7.2 ML Classification Pipeline
FR-6	Prediction Confidence Visualization	2.4 Results Processing and Display, 5.1 Mobile Application Components, 6.0 Human Interface Design
FR-7	Comparison with Training Data	5.3 Machine Learning Components, 7.2 ML Classification Pipeline
FR-8	Safety Information and Disclaimers results	2.4 Results Processing and Display, 5.1 Mobile Application Components, 6.3 Screen Objects and Actions
FR-9	History and Lesion Tracking	2.5 Analysis History and Data Management, 4.0 Data Design, 5.4 Data Management Components, 6.3 Screen Objects and Actions
FR-10	Data Management	4.0 Data Design, 5.4 Data Management, 8.0 Security and Privacy Design
FR-11	Application Interface	5.1 Mobile Application Components, 6.0 Human Interface Design

NFR-1	Performance	9.1 Performance Targets, 9.2 Optimization Strategies
NFR-2	Scalability	3.0 System Architecture, 9.2 Optimization Strategies
NFR-3	Accuracy	5.3 Machine Learning Components, 7.2 ML Classification Pipeline, 9.1 Performance Targets
NFR-4	Reliability	3.0 System Architecture, 7.3 Client-Server Communication, 9.1 Performance Targets
NFR-5	Usability	6.0 Human Interface Design
NFR-6	Security	3.3 Design Rationale, 8.0 Security and Privacy Design
NFR-7	Maintainability	3.2 Decomposition Description, 5.0 Component Design

Table 10.1 - Requirements Traceability Matrix

11.0 References

<https://approov.io/knowledge/should-we-use-https-or-tls-certificate-pinning-to-protect-mobile-apps>

<https://www.sciencedirect.com/topics/computer-science/mobilenetv2>

<https://medium.com/@kanishks772/stop-storing-files-in-your-database-youre-probably-doing-it-wrong-d6c26db5e1b5>

12.0 Appendices

12.1 Appendix A: System Glossary and Acronyms

- API (Application Programming Interface): Software allowing a mobile app to communicate with a server.
- CNN (Convolutional Neural Network): A type of machine learning model used for analyzing images.
- ISIC / HAM10000: Skin lesion image datasets our system will use to train and test the machine learning model.

- HTTPS / TLS: Security protocols that encrypt data sent between mobile app and server.
- SDD: Software Design Document
- SRS: Software Requirements Specification

12.2 Appendix B: System Error Messages

- | Error Condition | User Message | System Action |
- | Server or network connection lost | “Server Unable to be Reached” | Displays message; stops processing without giving a prediction |
- | Uploaded image blurry or unreadable | “Image quality is too low. Please submit another image” | Rejects photo and asks user to retake it |
- | Incorrect login credentials entered | “Invalid username or password” | Prompts user to re-enter login details |
- | Guest user trying to view saved history | “History is only available for registered accounts” | Prompts user to log in or create an account |
- | Analysis takes longer than 5 seconds | “Analysis timed out. Please try again” | Cancels request and allows user to try again |

12.3 Appendix C: Screen Reference Index

- Screen 1: Welcome & Login Screen (Login, Guest Mode, Register)
- Screen 2: Home Screen (Capture photo, upload image, view menu)
- Screen 3: Image Review Screen (Check image framing and lighting)
- Screen 4: Results Screen (View benign/malignant prediction, confidence score, disclaimer)
- Screen 5: History Screen (View past results for logged-in users)

12.4 Appendix D: Technology Stack

- Mobile Frontend: Android (Java)
- Preprocessing Tools: Python, OpenCV, Keras
- Machine Learning Models: PyTorch / Keras (CNNs like MobileNetV2, ResNet50, EfficientNetB0)
- Datasets: ISIC, HAM10000
- Backend Server: Python REST API Gateway
- Database & Storage: PostgreSQL (user accounts) and Firebase/Cloud Storage (images)
- Security: HTTPS / TLS encryption